

Network and IoT Security | NIOsec

Summary

CONTENTS

1. Introduction	2
2. TLS	2
2.1. Specification	2
2.1.1. Record protocol	3
2.1.2. Subprotocols	3
2.1.3. Handshake	3
2.2. Modern TLS Variants	3
2.2.1. DTLS	3
2.2.2. QUIC	4
2.3. Attacks	4
2.3.1. Heartbleed	4
2.3.2. Syn Flooding	4
2.3.3. TCP Reset	4
2.3.4. FREAK	5
3. Cryptography	5
3.1. Protocols	5
3.2. Standards	5
3.3. Symmetric-Key Cryptography	5
3.3.1. Block Cipher	6
3.3.2. Stream Cipher	7
3.3.3. Overall Construction	7
3.4. Asymmetric Cryptography	7
3.4.1. Signature Schemes	7
3.4.2. Public Key Infrastructure (PKI)	8
3.4.3. Signatures vs MACs	8
3.4.4. Establishing a shared key	8
3.4.5. Asymmetric Encryption	9
3.4.6. Overall Construction	9
3.5. Maths <3	9
3.6. Quantum Cryptography	10
3.6.1. Quantum Key Distribution	10
3.6.2. Polarization of Light	10

1. INTRODUCTION

OSI Stack:

<i>Communication layers</i>	<i>Security protocols</i>
Application layer	Web Application Security, VoIP Security, SW Security
Transport layer	TLS
Network layer	IPsec
Data Link layer	L2TP, IEEE 802.1X, IEEE 802.1AE, IEEE 802.11i (WPA2)
Physical layer	Quantum Key Exchange

The CIA triad is a foundational information-security model stating that systems should protect:

Confidentiality: Keeping information secret

Integrity: Keeping information correct and unaltered

Availability: Ensuring information and systems remain accessible

Additional desired properties include:

Authenticity: Adversary cannot claim to be someone else

Accountability/Non-repudiation: Actions can be traced to actor

2. TLS

SSL and TLS are protocols for internet handshakes and encrypted transmission. Secure Socket Layer (SSL) came first, then after v3.0 it became Transport Layer Security (TLS), currently v1.3. People still use SSL as a term, even though technically it's now TLS.

TLS 1.2: Introduced authenticated encryption, which is the most modern form of encryption. It's a more flexible protocol, lots of TLS extensions, the hashing function has been improved.

TLS 1.3: The protocol has been redesigned to make it faster. A lot of the old ciphers have been removed to make it more secure. It only supports authenticated encryption (AEAD).

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256

TLS_DHE_RSA_WITH_AES_128_GCM_SHA256

TLS_DHE_RSA_WITH_AES_256_GCM_SHA384

Encryption	Key		_Signature_	WITH	Bulk		_Message
protocol	exchange	algorithm			encryption	Authentication	
	algorithm				algorithm	Code (MAC)	

TODO:

W1 slides 22

2.1. SPECIFICATION

Handshake	Cipher Change Spec	Alert	Application Data
-----------	--------------------	-------	------------------

Record Protocol

2.1.1. Record protocol

All TLS packets are sent using the record protocol. This header is attached to every single message.

Header			Data
Type (1B)	Version (2B)	Length (2B)	...

Type: which kind of TLS message is sent (20,21,22,23)

Version: which TLS version is used

Length: how long is the payload of the message

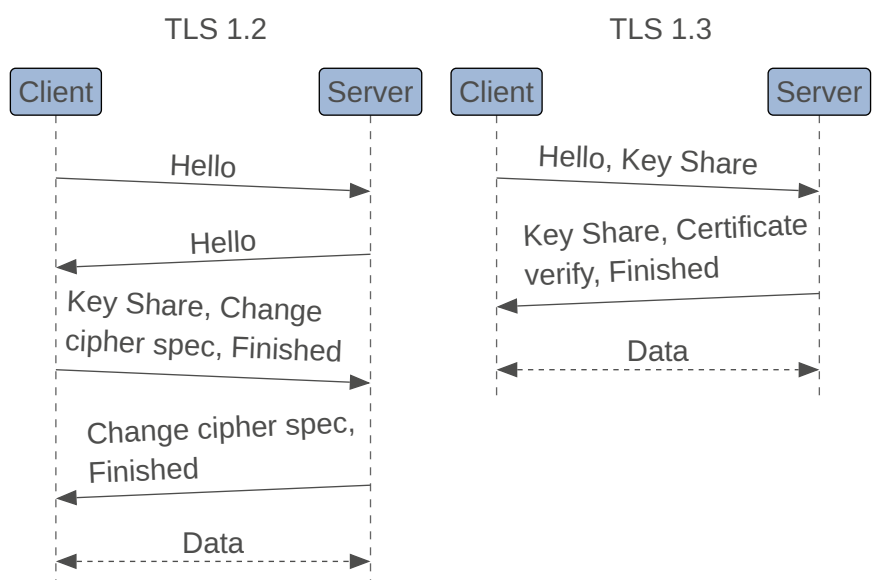
2.1.2. Subprotocols

Protocol	Definition
20 ChangeCipherSpec	The sender has sufficient information and is switching to encryption or to a new cipher suite (sent very rarely). By starting encryption for the first time, it is saying that the handshake has been successful, and encryption can start
21 Alert	An SSL-level alert notification (example close_notify). Not necessarily an error. Example: problem with the handshake, received data length is wrong, warning.
22 Handshake	Messages sent right at the beginning (example ServerKeyExchange). Eg. To establish a cipher, to establish our keys
23 Application Data	Opaque application data. Any applications data once we start the encryption is going to be sent under protocol 23

2.1.3. Handshake

The TLS handshake protocol is used to establish parameters for the remainder of the session. It must:

- Agree ciphers and protocols
- Establish shared secrets
- Authenticate server and client
- Be robust to tampering and attacks



2.2. MODERN TLS VARIANTS

2.2.1. DTLS

- TLS on top of UDP
- Used for WebRTC, VoIP

WebRTC/VoIP

- Does not provide ordering of packets
- Packets may drop, retransmission is delegated to application

DTLS

UDP

IP

2.2.2. QUIC

- TLS 1.3 + UDP
- Provides reliable communication
 - Stateful
 - Multiple streams
- Used for HTTP/3, RPC

HTTP/3, RPC

QUIC

UDP

IP

2.3. ATTACKS

2.3.1. Heartbleed

The Heartbeat extension is a keep-alive feature of TLS. The `heartbeat_request` message includes payload length, payload and padding fields. The Heartbleed bug is an implementation flaw in that extension.

- 1) Send message that indicates the maximum payload length (64 KB) that only includes the minimum payload (16 bytes)
- 2) Receive almost 64 KB of random memory that the server returns
- 3) Profit (look for private keys, auth cookies, etc.)

Countermeasures: Update OpenSSL

2.3.2. Syn Flooding

Idea: Fill the TCB (Transmission Control Block) queue storing the half-open (never ACKed) TCP connections so that there will be no space to store TCB for any new half-open connections. Basically the server cannot accept any new SYN packets.

- 1) Use random source IP addresses; otherwise the attacks may be blocked by the firewalls.
- 2) Spam TCP requests
- 3) Profit (sleep well knowing you're a menace to society)

The SYN+ACK packets sent by the server may be dropped because forged IP address may not be assigned to any machine. If it does reach an existing machine, a RST packet will be sent out, and the TCB will be dequeued.

Countermeasures: SYN Cookies

- After a server receives a SYN packet, it calculates a keyed hash (H) from the information in the packet using a secret key that is only known to the server.
- This hash (H) is sent to the client as the initial sequence number from the server. H is called SYN cookie.
- The server will not store the half-open connection in its queue.
- If the client is an attacker, H will not reach the attacker.
- If the client is not an attacker, it sends H+1 in the acknowledgement field.
- The server checks if the number in the acknowledgement field is valid or not by recalculating the cookie.

2.3.3. TCP Reset

Spoof RST Packet to break up a TCP connection between Alice and Bob.

The following fields need to be set correctly: Source IP address, Source Port, Destination IP address, Destination Port, Sequence number (within the receiver's window)

2.3.4. FREAK

- Man-in-the-middle (MitM) downgrades used TLS cipher suite to use export RSA
- MitM then factors 512 bit RSA to get session key

TODO:

https://ciphersuite.info/cs/TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256/

3. CRYPTOGRAPHY

Parties: The abstract entities in cryptographic protocols (servers, switches, humans)

Attacker model: Defines the capabilities of the attacker (computational limits, corruption)

Secure channel: Provides Confidentiality (we allow length leakage) and Authenticity (we don't care about liveness)

Authentic channel: Only provides Authenticity

3.1. PROTOCOLS

TODO:

W2 slides 18

3.2. STANDARDS

Don't roll your own crypto, except if you work at FAANG, they shall perish.

National Institute of Standards and Technology (NIST) publishes

- Special Publications (SP), guidelines and recommendations
- Federal Information Processing Standards (FIPS), standards for US government

Internet Society (ISOC)

- Organizes working groups for TCP/IP
- Publishes RFCs

3.3. SYMMETRIC-KEY CRYPTOGRAPHY

Insecure channel: Vulnerable to MITM Attacks

Shared key: Secure means the adversary has no information about the shared key. Establishing a shared key is non-trivial

- Pre-shared key (Code book)
- Quantum key exchange
- Public Key Cryptography

Message Authentication Codes (MAC): Provides authentic channel over an insecure channel (a short piece of information used for authenticating and integrity-checking a message, like a signature but symmetric). Vulnerable to length-extension-attacks.

Cryptographic hash function: Should be collision-resistant and one-way. **Good:** SHA2, SHA3, **Bad:** MD5, SHA-1

Symmetric Encryption: Provides secure channel over an authentic channel

Hash-based MAC (HMAC): Fixes length-extension-attack vulnerability.

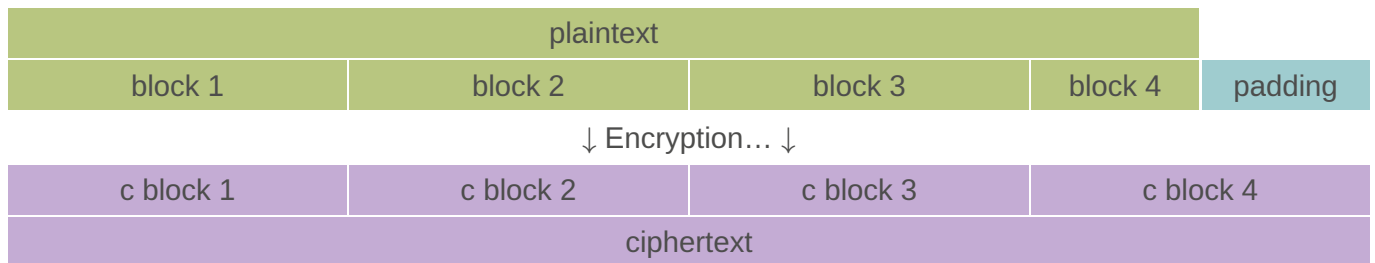
$$\text{Hash}(\text{key} \oplus \text{opad} \mid \text{Hash}(\text{key} \oplus \text{ipad} \mid m))$$

Block Size: Size of transformed/compressed data in the hash function for each iteration.

Message Digest Size: The output size of the hash function, also called hash value.

3.3.1. Block Cipher

Block ciphers take an input of a fixed size and return an output of the same size



- Block ciphers attempt to hide the transformation from message to ciphertext through confusion and diffusion
- Most block ciphers are SP-Networks

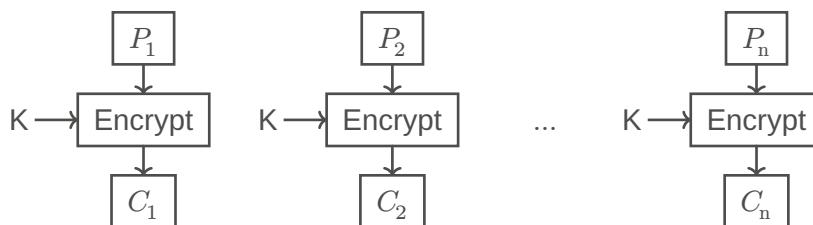
The Advanced Encryption Standard (AES) is an SP-Network

- Almost everything uses AES
- There are others (e.g. Feistel Ciphers)

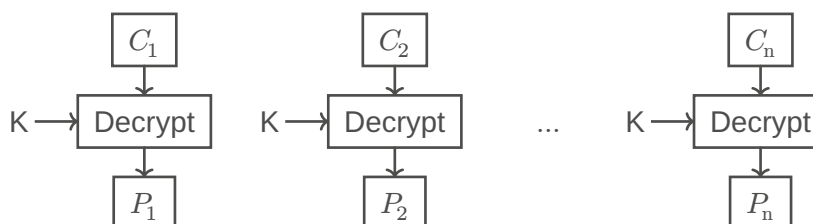
3.3.1.1. Electronic Code Block (ECB)

- Just encrypt each block one after another with same key
- Weak to redundant data divulging patterns
- Electronic codebook is not recommended!

Encryption



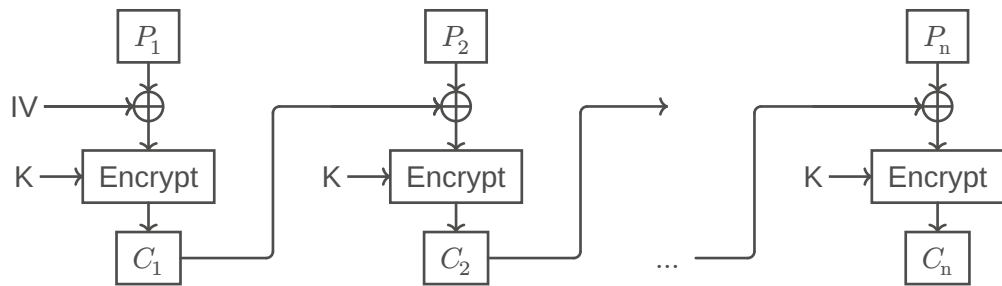
Decryption



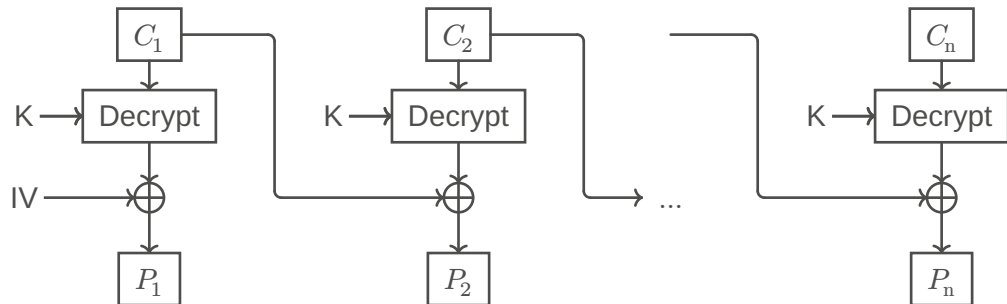
3.3.1.2. Cipher Block Chaining (CBC)

- XOR the IV with the first input, then XOR the output of each cipher block with the next input
 - Not parallelizable
 - It is better than ECB but not perfect

Encryption



Decryption



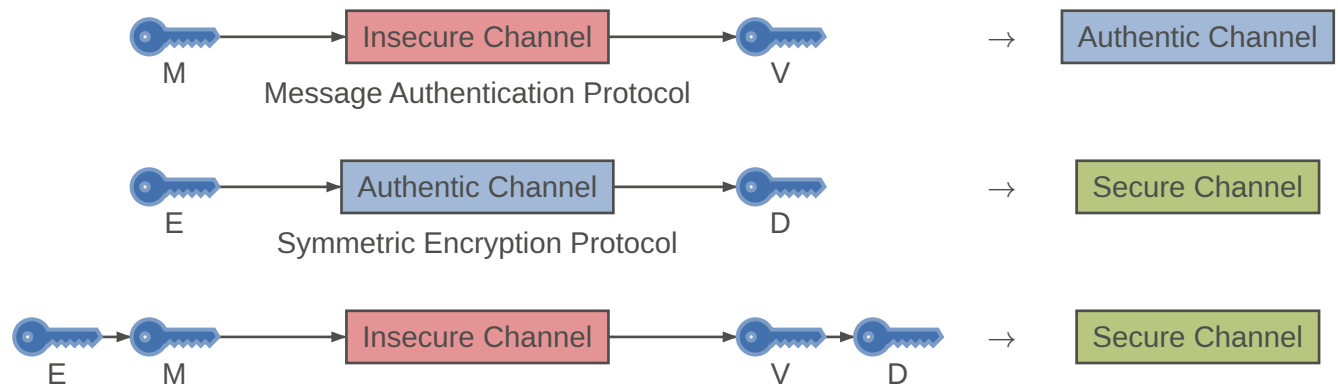
3.3.2. Stream Cipher

Inputs: Secret key + Public nonce (aka IV)

We can approximate a one-time pad by generating an infinite pseudo-random keystream

- Stream ciphers work on messages of any length (no blockification required)
- The nonce guarantees that each keystream is unique, even if the same key is reused

3.3.3. Overall Construction



Authenticity: E.g. HMAC-SHA256

Confidentiality: E.g. AES-CBC

Authenticity & Confidentiality: E.g. HMAC-SHA256-AES-CBC, AES-GCM

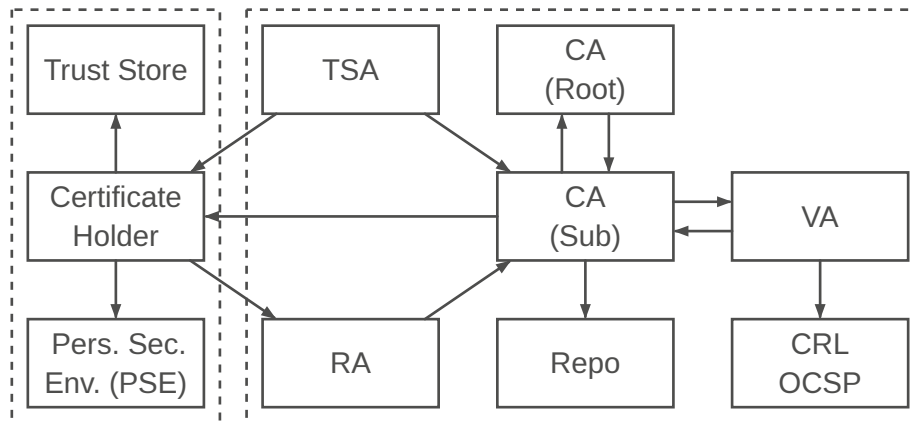
3.4. ASYMMETRIC CRYPTOGRAPHY

3.4.1. Signature Schemes

Private key used to **sign** messages. **Public key** used to **verify** signature. Signatures provide authenticity and non-repudiation. Examples: RSA, DSA, ECDSA

Signing is (often) computationally heavy. Some signature schemes natively only support small messages (naive RSA). Solution: Sign the hash of the message instead of the message itself

3.4.2. Public Key Infrastructure (PKI)



Each party has a private key and every other party knows their public key, e.g. via a registry. X.509 certificates and Certificate Authorities (CA) are used.

PKI provides an authentic channel over an insecure channel.

3.4.3. Signatures vs MACs

Signatures	MACs
Authenticity, non-repudiation	Authenticity only (key shared)
Slow (asymmetric)	Fast (symmetric)
Often: Use with long term keys	Often: Used with ephemeral keys

3.4.4. Establishing a shared key

Has to be established over an authentic channel.

Good practice: Ephemeral keys, randomly generated for each session. Provides Independence between sessions and limits impact if compromised.

Problem: Authenticity, you need to know with whom we agree on a key, otherwise MitM possible. One cannot use MACs (no shared key available).

Key Exchange Protocol (KEX): Both parties contribute to the key, often the same protocol for both parties (e.g. DH KEX)

Key Encapsulation Mechanism (KEM): One party generates the key, Key is wrapped (= encrypted) and sent to the other party

TODO:

W2 diagram slides 46

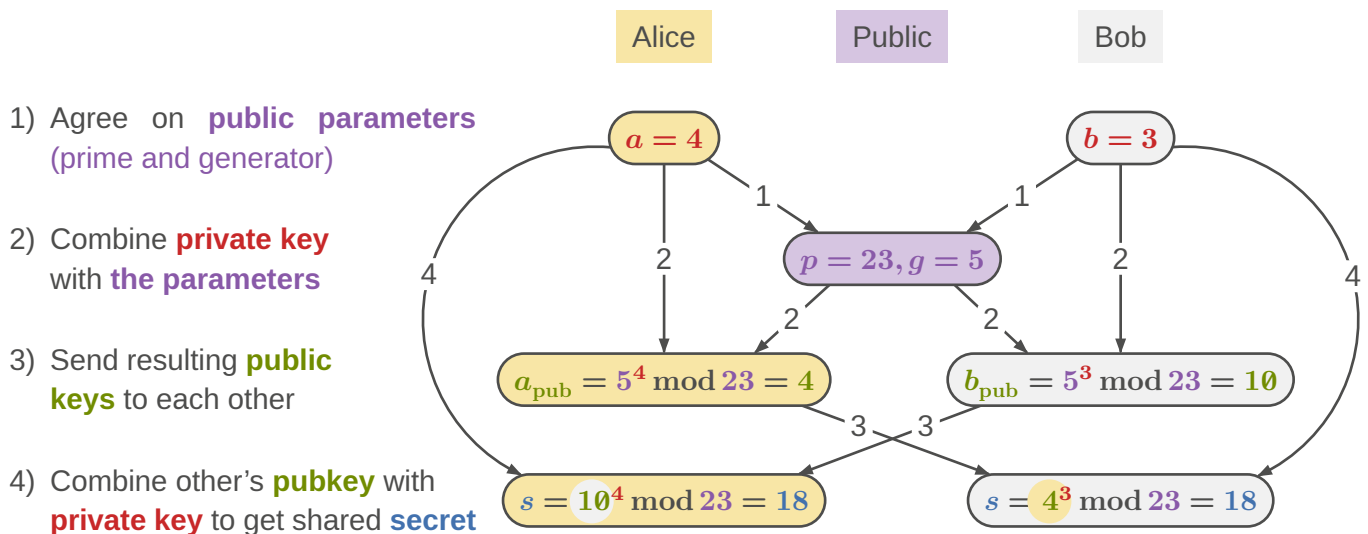
3.4.4.1. Diffie-Hellmann

- With Diffie-Hellman, two parties can jointly agree a shared secret over an insecure channel
- Every communication handshake on the internet is powered by DH
- We are exchanging some parts of the mathematical key and then we secretly create the key ourselves

Process:

- p is usually 4096 or 6144 bits
- g is a primitive root of p
- **private keys** are values between 1 and p
- The **shared secret** (often called the pre-master secret) serves as the foundation for deriving all subsequent session keys.

- The raw shared secret is not used directly for encryption, as it is typically a very large integer (e.g., 4096 bits in RSA or DH) and may not have uniform entropy.
- We derive a master secret using a hashed-key derivation function (HKDF), for example the SHA-256 hash function
- The only way to find **a** or **b** is to solve the Discrete Logarithm Problem.



Elliptic-Curve Diffie Hellman (ECDH) is becoming the standard nowadays due to shorter keys.

3.4.5. Asymmetric Encryption

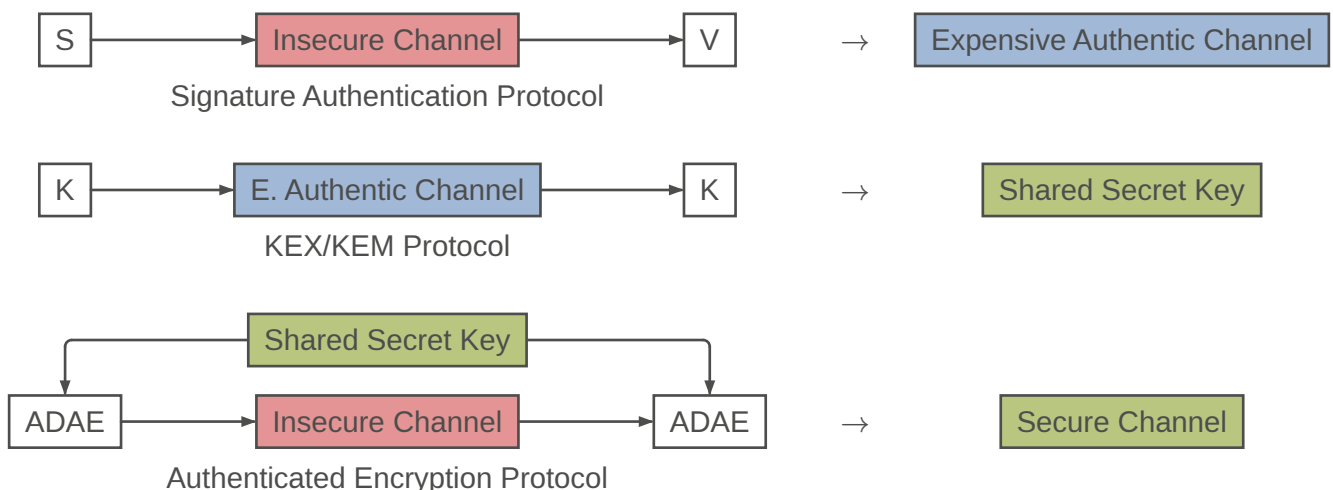
- Public key used to encrypt messages
- Private key used to decrypt messages

E.g. RSA (factorization) or ElGamal (discrete logarithm)

TODO:

W2 slides 48,51

3.4.6. Overall Construction



3.5. MATHS <3

TODO:

W2, merge with DigCod

3.6. QUANTUM CRYPTOGRAPHY

Quantum Cryptography: Using quantum effects to do cryptography, All parties have access to quantum systems. Example: Quantum Key Exchange BB84

Post-Quantum Cryptography: Cryptography secure even if quantum computers exist, the adversary has access to a quantum computer, honest parties are (often) classic. Example: ML-KEM

TODO:

W3, S9 key length

A big enough quantum computer could

- **break** known **asymmetric** schemes (Shor algorithm) e.g. RSA (factoring) and ElGamal, DH, ECDSA (discrete log.)
- **not break** **symmetric** schemes e.g. Hash functions (SHA-2) and AES-256

3.6.1. Quantum Key Distribution

TODO:

W3, S14..18

Security based on physical properties of quantum systems instead of hard math problems.

3.6.2. Polarization of Light

TODO:

W3, S18..

